

Lab 03 — Hash, Merkle tree & Digital Signatures

Bài thực hành 03 — Băm, cây Merkle & Chữ ký số

Time / Thời lượng: 50 minutes in class / 50 phút trên lớp. **Pass condition / Điều kiện đạt:** Lab 3.2 runs all checks + demo Lab 3.3 to the TA. / Lab 3.2 chạy qua toàn bộ kiểm tra + demo Lab 3.3 cho trợ giảng.

Setup / Chuẩn bị:

```
python3 --version      # >= 3.8
```

```
pip install eth-account # only needed for Lab 3.3 / chỉ cần cho Lab 3.3
```

Lab 3.1 — Hash properties & toy Proof-of-Work (10')

EN: Run and observe; then answer the questions in your submission file. **VI:** Chạy, quan sát; sau đó trả lời câu hỏi vào file bài nộp.

```
import hashlib, time
```

```
# (a) Avalanche effect / Hiệu ứng tuyết lở
```

```
print(hashlib.sha256(b"Blockchain 2026").hexdigest())
```

```
print(hashlib.sha256(b"blockchain 2026").hexdigest())
```

```
# (b) Toy PoW: find nonce so that sha256(data+nonce) starts with k zeros
```

```
# PoW đồ chơi: tìm nonce sao cho sha256(data+nonce) bắt đầu bằng k số 0
```

```
def mine(data: bytes, k: int) -> tuple[int, float]:
```

```
    target = "0" * k
```

```
    nonce, t0 = 0, time.time()
```

```
    while True:
```

```

h = hashlib.sha256(data + str nonce).encode()).hexdigest()

if h.startswith(target):

    return nonce, time.time() - t0

nonce += 1

for k in range(1, 7):

    nonce, dt = mine(b"block#1|txroot=abc|", k)

    print(f"k={k} nonce={nonce:>9} time={dt:.3f}s")

```

Q1 / Câu hỏi 1: Each extra leading zero multiplies expected work by $\approx$ how much? Why? / Mỗi số 0 thêm vào nhân khối lượng tính toán kỳ vọng lên $\approx$ bao nhiêu lần? Vì sao? **Q2 / Câu hỏi 2:** Verifying your found nonce takes how many hash calls? What does this say about PoW? / Kiểm tra nonce tìm được tốn mấy lần gọi băm? Điều đó nói gì về PoW?

Lab 3.2 — Merkle tree (20')

EN: Complete the three **TODOs** in `merkle_starter.py`, then run it — all **CHECK** lines must print **OK**. **VI:** Hoàn thành ba **TODO** trong `merkle_starter.py`, chạy lại — mọi dòng **CHECK** phải in **OK**.

Functions to implement / Các hàm cần cài:

1. `merkle_root(leaves)` — build the tree bottom-up; duplicate the last node when a level has odd length (Bitcoin rule). / dựng cây từ dưới lên; nhân đôi nút cuối khi tầng lẻ (quy tắc Bitcoin).
2. `merkle_proof(leaves, index)` — return the list of (`sibling_hash`, `is_left`) pairs from leaf to root. / trả về danh sách cặp (**băm anh em, nằm bên trái?**) từ lá lên gốc.
3. `verify_proof(leaf_hash, proof, root)` — recompute upward and compare. / tính ngược lên và so sánh.

Q3 / Câu hỏi 3: For $n = 1,000,000$ transactions, how many hashes does one proof contain? / Với $n = 1.000.000$ giao dịch, một proof chứa bao nhiêu giá trị băm? **Q4 / Câu hỏi 4:** Explain one real system that uses exactly this mechanism (SPV, airdrop claim, proof-of-reserves...). / Nêu một hệ thống thực dùng đúng cơ chế này.

Lab 3.3 — Sign, verify & recover with eth-account (15')

```
from eth_account import Account

from eth_account.messages import encode_defunct

acct = Account.create()          # NEVER use this key for real funds

print("address:", acct.address)   # TUYỆT ĐỐI không dùng khóa này cho tiền thật

msg = encode_defunct(text="I attended Session 3 / Tôi đã học Buổi 3")

sig = Account.sign_message(msg, acct.key)

print("r,s,v:", hex(sig.r), hex(sig.s), sig.v)

# Recover the address from the signature alone / Khôi phục địa chỉ chỉ từ chữ ký

who = Account.recover_message(msg, signature=sig.signature)

print("recovered:", who, "| match:", who == acct.address)

# Tamper 1 character / Sửa 1 ký tự

bad = encode_defunct(text="I attended Session 3 / Tôi đã học Buổi 4")

print("tampered ->", Account.recover_message(bad, signature=sig.signature))
```

Tasks / Nhiệm vụ:

1. Run twice with the same message — is the signature identical? Which RFC explains this? / Chạy hai lần cùng thông điệp — chữ ký có giống nhau không? RFC nào giải thích điều này?
2. Show the TA: the tampered message recovers a *different* address. Explain why this proves integrity. / Cho trợ giảng xem: thông điệp bị sửa khôi phục ra địa chỉ *khác*. Giải thích vì sao đây là bằng chứng toàn vẹn.
3. **Bonus / Nâng cao:** sign the same text with MetaMask (`personal_sign` in the browser console on a test profile) and recover it with `Account.recover_message` — bridging lab and wallet. / ký cùng đoạn văn bằng MetaMask rồi khôi phục bằng `Account.recover_message` — nối lab với ví thật.

Submission / Nộp bài

EN: Push `merkle_starter.py` (completed) + `answers.md` (Q1–Q4 + Lab 3.3 outputs) to your class Git repo before 23:59 today. **VI:** Đẩy `merkle_starter.py` (đã hoàn thành) + `answers.md` (Q1–Q4 + kết quả Lab 3.3) lên repo Git của lớp trước 23:59 hôm nay.

=====

```
#!/usr/bin/env python3
"""Lab 3.2 – Merkle tree (starter).

Hoàn thành 3 TODO rồi chạy:  python3 merkle_starter.py
Mọi dòng CHECK phải in OK.   All CHECK lines must print OK.

Quy ước (theo Bitcoin) / Convention (Bitcoin rule):
- Lá = sha256(dữ liệu giao dịch).           Leaf = sha256(tx bytes).
- Cha = sha256(trái + phải) trên byte digest. Parent = sha256(left +
right) over raw digests.
- Tầng lẻ: nhân đôi phần tử cuối.           Odd level: duplicate the
last element.
"""
import hashlib

def H(b: bytes) -> bytes:
    return hashlib.sha256(b).digest()

def merkle_root(leaves: list[bytes]) -> bytes:
    """TODO 1: dựng cây từ dưới lên, trả về băm gốc.
    Build bottom-up, return the root digest."""
    raise NotImplementedError

def merkle_proof(leaves: list[bytes], index: int) -> list[tuple[bytes,
bool]]:
```

```

    """TODO 2: trả về [(sibling_digest, sibling_is_left), ...] từ lá lên
    gốc.

    Return the sibling path from leaf `index` up to the root."""
    raise NotImplementedError

def verify_proof(leaf_hash: bytes, proof: list[tuple[bytes, bool]], root:
bytes) -> bool:
    """TODO 3: tính ngược lên gốc rồi so sánh. Recompute upward and
    compare."""
    raise NotImplementedError

# -----
checks
if __name__ == "__main__":
    txs = [f"tx{i}: A->B {i} coin".encode() for i in range(8)]
    leaves = [H(t) for t in txs]
    root = merkle_root(leaves)

    print("root:", root.hex())

    # CHECK 1: proof đúng cho mọi lá / valid proof for every leaf
    ok = all(verify_proof(leaves[i], merkle_proof(leaves, i), root) for i
in range(8))
    print("CHECK 1 (all 8 proofs valid):", "OK" if ok else "FAIL")

    # CHECK 2: proof có đúng log2(8)=3 phần tử / proof has exactly 3
elements
    print("CHECK 2 (proof length == 3):", "OK" if len(merkle_proof(leaves,
4)) == 3 else "FAIL")

    # CHECK 3: lá bị sửa phải trượt / a tampered leaf must fail
    fake = H(b"tx4: A->B 999999 coin")
    print("CHECK 3 (tampered leaf fails):",
        "OK" if not verify_proof(fake, merkle_proof(leaves, 4), root)
else "FAIL")

```

```
# CHECK 4: số lá lẻ (7) vẫn chạy / odd leaf count (7) still works
l7 = leaves[:7]
r7 = merkle_root(l7)
print("CHECK 4 (odd count works):",
      "OK" if all(verify_proof(l7[i], merkle_proof(l7, i), r7) for i in
range(7)) else "FAIL")
```